

第 16 讲：哪些经验应该成为“共同记忆”？

Shared Memory Governance, Selective Consolidation and Memory Lifecycle

从“是否共享”推进到“谁有资格进入群体的制度性记忆”

Shared Memory Governance = Admission + Maintenance + Audit

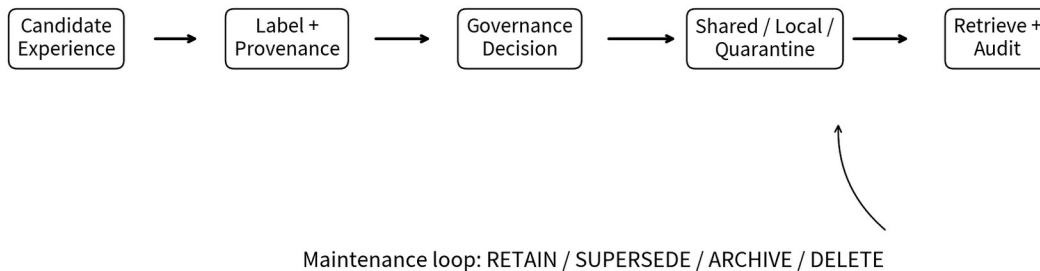


图 1 Shared Memory Governance 不是一个 retrieval 技巧，而是一套从准入、维护到纠错的生命周期制度。

1. 从第 15 讲继续：共享本身不是最终问题，治理才是

第 15 讲已经看到：Full-Pool 可能让有价值的经验传播得更快，也可能让 memory overlap、error correlation 和 wrong consensus 一起上升。于是下一步不能只问“共享多少”，而应该问：一条新 experience 到来时，它究竟有没有资格成为所有 Agent 都能检索到的共同记忆？

- 局部经验不一定适合全局传播：某个 Agent 的特殊约束、偏好或临时 workaround 可能只应该留在本地。
- 成功经验也可能已经过期：旧政策、旧 API、旧参数仍然可能产生高相似度检索，却把整个系统带向错误。
- 失败经验不是垃圾：它可能应该进入 quarantine / caution memory，而不是被当作成功示范。
- 共享池不是“无限硬盘”：预算有限时，重复经验会挤掉稀缺但重要的能力覆盖。

核心科研问题

Shared memory 应该被看作一种“institutional state”。真正的问题不是 how to remember，而是 who/what gets selected into persistent shared state, how it can be corrected, and how diversity is preserved.

2. Governance 和普通 Memory Management 有什么区别？

普通 memory management 经常关注 ADD / RETRIEVE / UPDATE / DELETE；governance 多了一层“制度性选择”：哪些记录允许跨 Agent 传播、哪些必须保留 provenance、哪些发生冲突时需要 supersede、哪些应该进入 quarantine、哪些最终应该被遗忘。

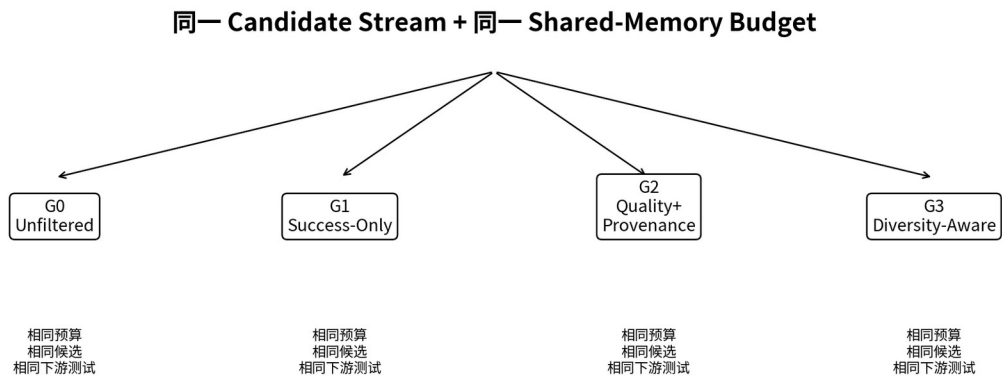
层次	典型问题	如果做错会怎样
Local memory	这个 Agent 自己要不要记住?	单体受影响
Shared memory	这条经验能不能被整个团队检索?	错误可能被放大
Governance	谁批准、按什么规则、怎样纠错/替换?	形成长期制度性偏差

3. 两阶段动作空间：Admission + Maintenance

- Admission: PROMOTE_SHARED——成为共享记忆。
- Admission: KEEP_LOCAL——有价值，但只适合原 Agent / 原上下文。
- Admission: QUARANTINE——可能有价值，但证据不足、存在冲突或需要复核。
- Admission: REJECT——明显低质量、越权、无关或不可迁移。
- Maintenance: RETAIN / SUPERSEDE / ARCHIVE / DELETE——随着时间和新证据持续维护。

一个常见误区是把 governance 退化成一个“质量分数排序”。但 stale、scope、contradiction、provenance 都不是单一分数能完整表达的；它更像数据库中的约束、版本控制与审核流。

4. 四种受控治理制度



只有 Governance Rule 改变 → 才能讨论“制度性记忆”因果效应

图 2 固定 Candidate Stream、预算和下游任务，只改变 Governance Rule。

条件	规则	主要缺陷/用途
G0 Unfiltered-Ingest	最近到来的记录直接进入共享池，预算满后截断	最简单基线；容易泄漏 local-only、保留 stale/矛盾记录
G1 Success-Only	只共享 outcome=success 的 experience	比全量好，但“成功”不等于可迁移、不过期、不冲突
G2 Quality+Provenance+Supersession	质量阈值 + share scope + provenance + stale filter + contradiction supersession	测试结构化治理本身的价值
G3 Diversity-Aware Governance	G2 基础上再加入 novelty、source/family coverage 与去重复	测试治理能否同时保护 shared-memory 的能力覆盖与异质性

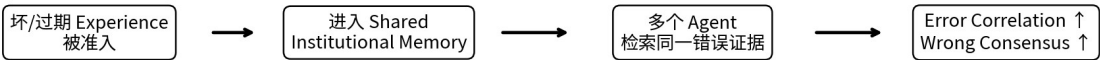
5. 为什么“成功经验”也可能不该共享？

一次成功只是“当时在那个上下文里奏效”。如果它依赖某个用户偏好、临时工具版本、特殊授权、一次性参数或者已经失效的政策，把它提升为 institutional memory 可能造成 negative transfer。

- Context-bound：只在原上下文成立 → KEEP_LOCAL。
- Unverified transferability：不知道能否跨任务复用 → QUARANTINE。
- Stale success：曾经成功但规则已改变 → SUPERSEDE / ARCHIVE。
- Duplicate success：大量同质成功经验会挤掉稀缺能力 → novelty/coverage gate。

6. 四类最值得监控的 Governance Failure

Failure	含义	下游风险
Bad-memory admission	错误、低质量或不可迁移经验进入共享池	多个 Agent 被同一错误记忆带偏
Stale propagation	旧版本经验长期可见	系统重复使用已经失效的规则
Contradiction persistence	新旧矛盾记录同时存活	检索结果依赖偶然排序，行为不稳定
Provenance collapse	来源、版本、适用范围不可追踪	无法审计，也无法准确撤回/修订



治理失败不是“记错一条”这么简单；共享记忆会把单点错误变成系统级共同盲点。

图 3 Shared memory 的危险在于：单条错误经验可能变成多个 Agent 的共同盲点。

7. 还要防第五类失败：Coverage Collapse

即使共享池里的每条记录都“正确”，如果 70% 都来自 policy 类、60% 都来自同一个 source agent，有限 memory budget 仍可能让 evidence / parameter 能力被挤出。于是 shared memory 看似质量很高，却失去能力覆盖。

因此第 16 讲同时统计 family/source entropy 与 redundancy。它们不是最终可靠性指标，但能告诉你共享池是否被重复、单一来源或热门任务占领。

8. Teaching Mock：治理好坏怎样传导到系统可靠性？

Regime	Admission precision	Local leak	Stale	Error ϕ	Majority err	Wrong consensus
Unfiltered	27.1%	14.6%	43.8%	0.276	30.0%	16.7%
Success-only	43.8%	10.4%	33.3%	0.284	26.7%	15.3%
Quality+Prov.	100.0%	0.0%	0.0%	0.062	4.0%	1.3%
Diversity-aware	100.0%	0.0%	0.0%	0.068	4.0%	1.3%

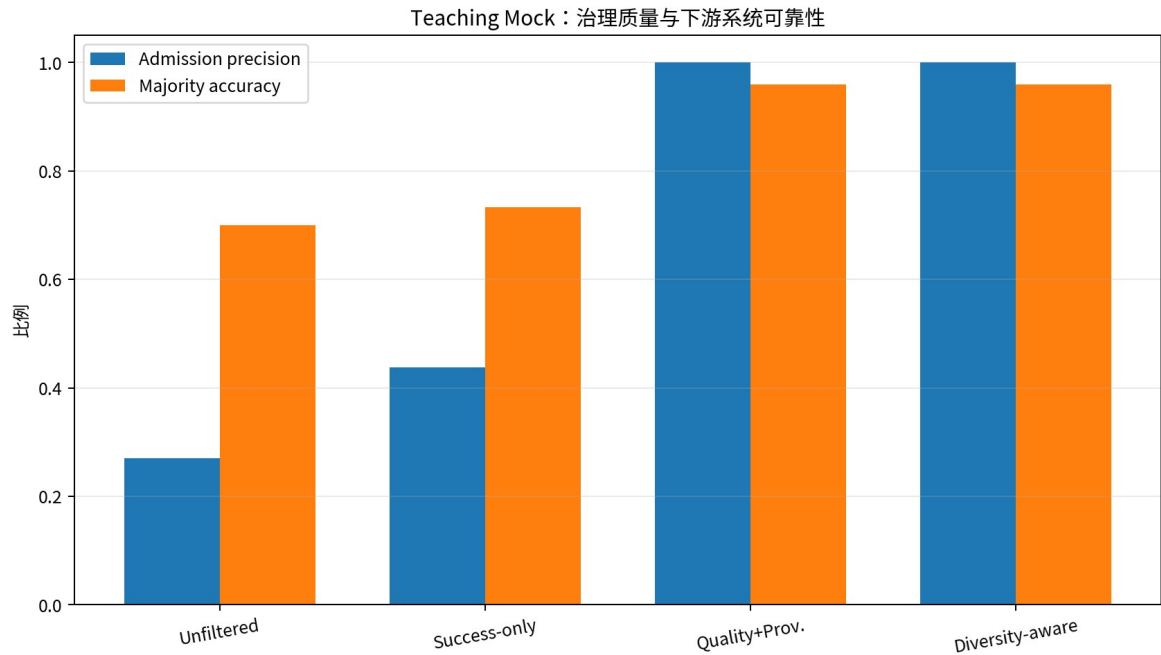


图 4 Teaching Mock：更干净的共享记忆准入，通常会降低共享错误证据造成的系统级失误。

重要边界

上表全部是人为合成 Teaching Mock。它只验证分析管线和因果直觉，不能当作任何真实模型或真实 memory architecture 的性能结论。

9. 一个零基础比喻：Shared Memory 像“医院公共病例规范库”

医生自己的笔记可以很灵活，但一旦某条经验进入整个医院都能查到的公共规范库，就必须要求来源、版本、适用范围、审核状态都明确。一次旧药物剂量如果被错误提升成公共规范，不再是一个医生犯错，而可能让所有科室一起犯错。Shared Memory Governance 就是在决定：什么能进规范库、什么只能留个人笔记、什么必须隔离复核、什么已经被新版本替代。

10. 当前研究已经走到哪里？

2026 年的研究已经从“有没有 memory”快速转向“怎样管理 memory”。Memory-R1 把 ADD / UPDATE / DELETE / NOOP 做成学习到的 memory manager；AMA 研究多粒度更新与一致性；Learning to Share 在并行 Agent 中学习哪些中间步骤值得进入 global memory；Shared Selective Persistent Memory 显示 full-history persistence 可能反而降低任务完成率；Governed Shared Memory 进一步把 scope、supersession、provenance 和 propagation 作为系统级原语。

但对我们这条主线而言，仍然要继续验证一个更具体的问题：治理制度能否在共享收益、错误相关与 Agent 异质性之间形成可重复的 trade-off，而不是只提升单任务 retrieval accuracy。

11. 真实实验最低闭环

- Step 1: 用第 15 讲的 frozen local banks 形成 candidate inventory。
- Step 2: 人工或 curator 标注 scope、outcome、quality、stale、contradiction、version、novelty。
- Step 3: 固定相同 shared-memory budget，离线跑四种 governance regime。
- Step 4: 冻结生成的共享池，再在完全相同 held-out tasks 上运行三个 Agent。
- Step 5: 保存 retrieved memory IDs，定位 bad-memory admission → shared retrieval → correlated error 的完整链。
- Step 6: 先做 6–10 case smoke test，再扩大；不要一开始就烧大量 API。

12. 本讲最低闭环能力

学完至少要能做到

- ① 区分 local memory management 与 shared-memory governance；② 为候选经验选择 PROMOTE / LOCAL / QUARANTINE / REJECT；③ 解释 stale、contradiction、provenance、coverage 为什么是系统级问题；④ 设计同候选流、同预算的治理消融；⑤ 把治理指标和下游 error correlation / collective reliability 连起来。